

Kestrel Home: service-request router, final report

1 October 2026 · repo: github.com/vaibhav375/kestrel-request-router · all numbers regenerated by `evaluate.py` and the scripts in `experiments/`

1. Summary

Ritu asked for a classifier that matches the vendor bot's labels at 90%. Those labels are the bot's own guesses, right **77%** of the time. A model that copies them reaches 91.8% agreement and routes no better than the bot. So the router was trained on **where each request was actually closed**.

	Vendor bot	Router
Sent to the team that closed it (Apr–Jun 2026, 2,135 unseen requests)	76.7%	86.0% (95% CI 84.5–87.5%)
Misrouted requests	23.3%	14.0% (40% fewer)
5-fold cross-validation (10,822 requests)	77.2%	86.3% ± 0.2
Macro-F1	78.0%	87.2%
Requests that state a need (84%)	87.8%	98.0%
Confident answers auto-routed (84% of volume)	–	98.4% right
Cost	Rs 3.2 lakh/year (~Rs 38 per request)	Rs 0 per request
Saving	–	about Rs 8.8 lakh a year

Expected score on the hidden test set: about **86% accuracy** (range 84–87%); macro-F1 about 87%; about 88% if scored against the bot's labels.

2. What the data showed

- **Payment remarks fool the bot.** Of 626 customers who mentioned paying (“paid on upi”), the bot sent 525 to Billing, and Billing closed **1**. Policy §3 says mentioning payment does not make a request Billing.
- **Purifier breakdowns** were sent to Consumables and closed by Repairs.
- **Two requests in one message: the last one decides** (542 of 549 messages, 98.7%).
- **~16% of requests state no need** (“please call me about my purifier”). Their outcomes are spread over all seven teams, and no column predicts them. This caps *any* text-only router at ~86%. An oracle that sees the answers reaches only 86.25%.
- **~1.6% of clear requests are closed by an unrelated team** at random (recording noise).
- **Data fixes:** team renames merged; legacy Zoho mojibake repaired; legacy resolution times were UTC (26% looked “resolved before created”), fixed +5:30; 126 inconsistent records kept and flagged.
- **Headcount:** the bot's labels overstate Repairs (+42/month), Billing (+24) and Consumables (+23), and understate Installs (–27), Returns (–25) and Warranty (–22).

3. The router

```
message
└─ clean: fix garbled legacy text, merge renamed teams
└─ parse: drop greeting, order no., sign-off, payment remark; the LAST request decides
└─ [B4] wording unknown? fix Hinglish + spelling, read again
    ├── states no need → guess by product (A1, labelled a guess) + 7-option question
    ├── wording known → team that closed most past requests with that wording (98% right)
    └── new wording → keyword rules → text model; unsure → clarifying question
└─ output: team · confidence · auto_route (C1) · reasons for the agent · alternatives
unsure requests → review queue → agent's answer via /feedback → learned at next retrain (B5)
```

FastAPI POST /route + one screen, POST /feedback, /health. No API key, no GPU, ~2 s to train, 176 KB model, worst case 3.7 ms per request. Starts on a clean machine; with no data it still starts and explains what to do. 44 automated tests.

4. How it was validated: four rounds

round	what was tested	outcome
1	~20 approaches on out-of-time, 5-fold CV and held-out wording: TF-IDF variants, embeddings, rules, lookup, local LLM (Qwen2.5-3B)	Lookup + last-request parsing best. The LLM scored 71%, worse than the bot. The copy-the-bot model scored 77%
2	11 challengers under a pre-committed protocol, rolling backtest (6,502 predictions), McNemar + Holm: fine-tuned transformer, gradient boosting, kNN, noise cleaning, ensembles	None significantly better; transformer identical on clear requests. Router stayed
3	Last untested signals (noise pattern, recent context, previous request, time) + operating levers	No exploitable signal. Auto-routing the confident 84% gives 98.4%
4	9 engineering ideas as switches, pre-committed protocol	Adopted: A1 product-based vague guess (+0.25, labelled a guess), B4 Hinglish/typo repair, B5 corrections loop, C1 auto-route flag. Rejected: fuzzy match, paraphrase lookup, 3-way vote (–10 pts), option reordering; embeddings +2.4 but below the bar for adding PyTorch

Round 4 effect (stress tests are synthetic):

	before round 4	final
Real data, out-of-time	85.7%	86.0%
Vague requests	20.2%	22.0%
Heavy typos (30% of words corrupted)	94.1%	97.6%
Hinglish (LLM-rewritten, noisy)	78.7%	82.0%
New phrasing after 3 agent-confirmed examples	89.8%	98.4%

5. Where it fails

Out-of-time it is wrong on **299 of 2,135 (14.0%)**:

- **262 vague requests.** Nobody can route these from the text; the router flags them and offers one question.
- **37 clear requests (2.1%)** closed by an unrelated team, mostly records the log itself marks as inconsistent.
- **On live traffic:** genuinely new wording is ~87% right on requests that state a need (range 70–98% by fold), so it is flagged when unsure and fed back through the corrections loop.

6. Money

Misrouting costs **Rs 14.7 lakh a year** today: 3,129 transfers × Rs 305, plus Rs 260 per repeat contact (policy \$4). Misrouted requests also take twice as long to close (16.6 h vs 8.2 h). The router cuts wrong-first-team from 23% to 14%, saving **Rs 5.6 lakh**. With the **Rs 3.2 lakh** licence that is **≈ Rs 8.8 lakh a year**, at **Rs 0 per request**: 700 requests × ~1.5 ms ≈ 1 s of CPU a month. If the intake question works: up to Rs 16 lakh (projection).

7. Recommendation

1. **Switch the bot off after a two-week side-by-side run.** Measure “closed by first team”: 77% today, target 86%+.
2. **Pilot the clarifying question on one channel** for vague requests. It is the only lever that can take accuracy beyond ~86%, towards ~98%, because it adds the information the message does not contain.
3. **Use real volumes, not bot labels, for headcount.**
4. **Retrain monthly** with agent-confirmed corrections; revisit embeddings if the review queue shows much new wording.

8. Honest limits

- Tested on history, not live traffic; the export is highly templated (132 distinct phrasings).
- Truth is the resolution log, which is itself ~2% noisy.
- The product-based vague guess (A1) is not statistically proven, is labelled a guess, and costs 0.3 points of macro-F1.
- The typo and Hinglish tests are synthetic; the Hinglish set is noisier than real customers.
- The keyword rules were written after reading the data; the fair robustness figure (87%) is the one reported.
- No login or rate limiting on the service; tested on macOS only.